



- What is an Image?
- Image Filtering
- Local Operations
 - Image Convolution
 - Linear filters
 - Smoothing
 - Gradient
 - Padding
 - Stride
 - Non linear filters
 - Bilateral filtering
 - Integral Image

What are images?



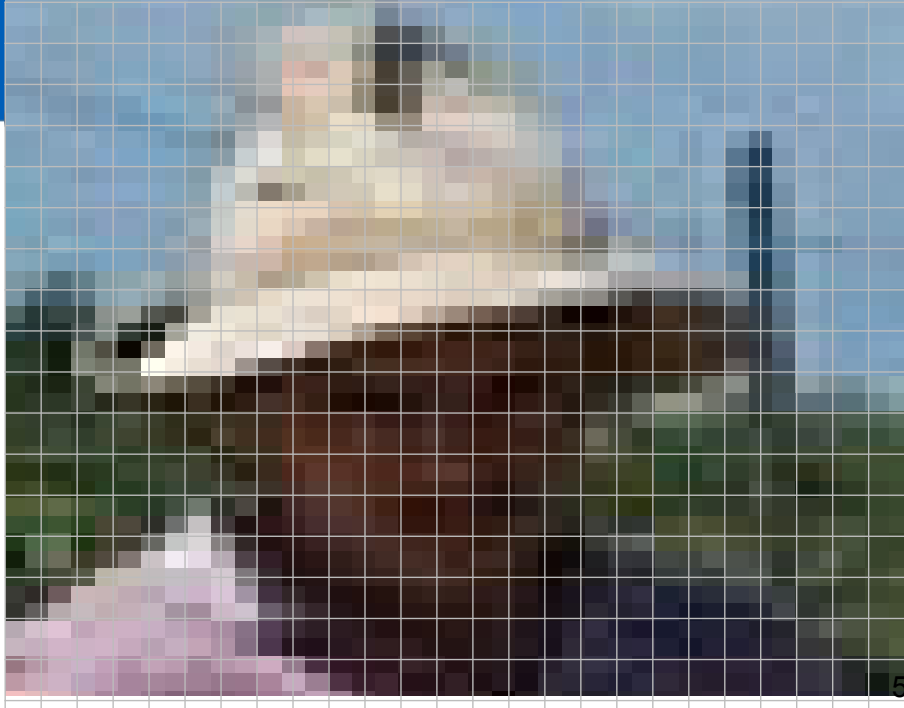
Prendiamo ad esempio una immagine a caso

What are images?



$$P = f(x, y)$$
$$f : \mathbb{R}^2 \Rightarrow \mathbb{R}$$

La posso vedere come una funzione 2D. Date le coordinate di riga e colonna (x,y) ottengo un valore. Che valore? Lo vedremo nel dettaglio successivamente ma nella forma più generale possibile è un numero reale (sì, ci possono essere anche valori non interi). Ma in realtà anche le coordinate di riga e colonna nella forma più generale potrebbero essere non interi e quindi ho una funzione da $\mathbb{R}^2$ a $\mathbb{R}$.



UNIVERSITÀ
DI PARMA

$P = f(x, y)$
 $f : \mathbb{R}^2 \Rightarrow \mathbb{R}$

1. Signal is **sampled** to obtain a function (r, c)
2. Result is also **quantized** given we need a discrete function
3. One or more **channels** for each pixel

$R(r, c), G(r, c), B(r, c)$
 $Luminance(r, c)$

Va comunque detto che la forma generale quasi mai è di interesse, per cui si quantizza e la mia funzione fornisce numeri interi. E in generale coordinate non intere sono di interesse in casi relativamente isolati.

Insomma si quantizza tutto.

Inoltre posso per ciascun punto di una immagine (**pixel**) avere più canali e quindi in realtà più funzioni.

Ad esempio per immagini a colori è abbastanza comune avere 3 canali, uno per ciascuna componente del colore (R rosso, G verde, B blu). Mentre per le immagini a toni di grigio tipicamente ne ho uno solo.

Ma esistono comunque situazioni anche più variegate (ci torneremo quando parleremo di formati immagine).

$$F(\text{img}) = \text{img}$$

Ora iniziamo a parlare di elaborazione immagini (non è di fatto visione artificiale).

La possiamo definire come applicazione di una funzione spesso del tipo $\mathbb{R}^2 \rightarrow \mathbb{R}^2$

In questa carrellata si vedono diversi esempi.

Quasi sempre il risultato dell'elaborazione di una immagine è una seconda immagine ma non necessariamente.



$$F(\text{img}) = \text{img}$$



$$F(\text{img}) =$$

0.1
0
0.8
0.9
0.9
0.9
0.2
0.4
0.3
0.6
0
0
0.1
0.5
0.9
0.9
0.2
0.4
0.3
0.6
0
0
0.1
0.9
0.9
0.2
0.4
0.3
0.3
0.3
0.3



$$F(\text{img1}, \text{img2}) = \text{img3}$$

- Filters
 - Mathematical functions that operate on the pixels
 - Often Image $\rightarrow$ Image
- Different classes
 - **Point operations:** result depends only on each pixel value
 - **Local operations:** result depends on each pixel + neighborhood values
 - **Global operations:** result depends on the values of all image pixels

Quindi l'elaborazione immagine è una funzione che opera sui pixel e spesso produce una seconda immagine come risultato.

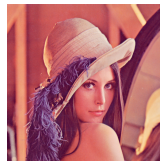
Si possono individuare 3 tipologie:

- Operazioni puntuali: la funzione applicata lavora sui singoli pixel, quindi il risultato dipende solo dal pixel in esame (esempi: modifica contrasto, luminosità...)
- Operazioni locali: il risultato dipende da un "intorno" più o meno ampio di ciascun pixel. Quindi prendo in esame un'area intorno al pixel in esame (esempio, riduzione rumore)
- Operazioni globali: la funzione di trasformazione prende in ingresso tutta l'immagine (esempio, trasformata di Fourier)

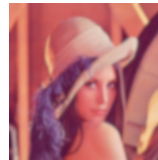
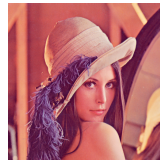
Image Filtering Examples



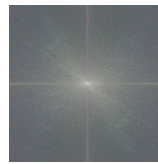
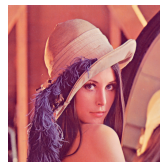
- Point operations:
 - Color to gray levels



- Local operations:
 - Running average (blur)



- Global operations:
 - Fourier transform



$$F(\text{img}) = \text{img}$$

- Result depends only on
 - Pixel value
 - Pixel Neighborhood

Come accennato la funzione di elaborazione dipende da un'area attorno al pixel in esame (vicinato).

- We already said that an image can be seen as a matrix
 - Each matrix element corresponds to a image point better known as **pixel**
 - Each pixel feature one or more values that allow to define its content
 - Color, luminance, transparency...
 - Not necessarily integer numbers...
- For following slides, we are going to deal with grey level only images
 - Each pixel must encode luminance only
 - Only 1 integer non negative number for each pixel
 - The higher the number, the brighter the pixel
 - 0 $\rightarrow$ black

- Image convolution is the main operation used for local image filtering
 - From $f[.,.] \rightarrow h[.,.]$
- $g[k,l]$ is the **Kernel** or Convolution Matrix and represents the pixel neighborhood weight
 - Kernel size

- **Linear operation**

- Symbol: *

- $h=g*f$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

Limitiamoci inizialmente a operazioni lineari

Indicando con $f[]$ la matrice immagine in ingresso e con $h[]$ l'immagine risultante, l'elaborazione sfrutta tipicamente una matrice di convoluzione $g[]$ o kernel con cui si pesano i pixel in ingresso.

Il kernel è una matrice di dimensioni significativamente minori di quelle dell'immagine da elaborare.

All'atto pratico, e come si vede nelle slide successive, io sovrappongo $g[]$ a $f[]$ centrandolo su ciascun pixel di $f[]$. Moltiplico i valori "sovrapposti" di $g[]$ e $f[]$ e li sommo tra di loro ottenendo il valore del corrispondente pixel di $h[]$. La formula in esempio spiega come calcolare il valore di h alle coordinate (m,l)

Local Operations



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	90	90	90	90	90	0
0	0	0	0	90	90	90	90	90	0
0	0	0	0	90	90	90	90	90	0
0	0	0	0	90	0	90	90	90	0
0	0	0	0	90	90	90	90	90	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} g[.,.]$$

Credit: S. Seitz

In questo caso il kernel è una matrice 3x3 composta da valori tutti pari a 1/9 (per semplicità uso un moltiplicatore esterno da 1/9 e una matrice di 1)

Local Operations



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	90	90	90	90	90	0
0	0	0	0	90	90	90	90	90	0
0	0	0	0	90	90	90	90	90	0
0	0	0	0	90	0	90	90	90	0
0	0	0	0	90	90	90	90	90	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[\cdot, \cdot]$

0	10								

$$h[m, n] = \sum_{k, l} g[k, l] f[m+k, n+l]$$

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} g[\cdot, \cdot]$$

Credit: S. Seitz

Local Operations



0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	0	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

$f[\cdot, \cdot]$

	0	10	20					

$$h[m, n] = \sum_{k, l} g[k, l] f[m+k, n+l]$$

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} g[\cdot, \cdot]$$

Credit: S. Seitz

Local Operations



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30					

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} g[.,.]$$

Credit: S. Seitz

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30	30				

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$g[.,.]$

Credit: S. Seitz

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30	30				

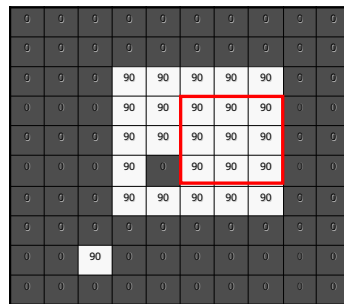
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

$\frac{1}{9}$
 $g[.,.]$

1	1	1
1	1	1
1	1	1

Credit: S. Seitz

UNIVERSITÀ
DI PARMA

[illegible]

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \mathbf{g}[\cdot, \cdot]$$

Credit: S. Seitz

Local Operations



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} g[.,.]$$

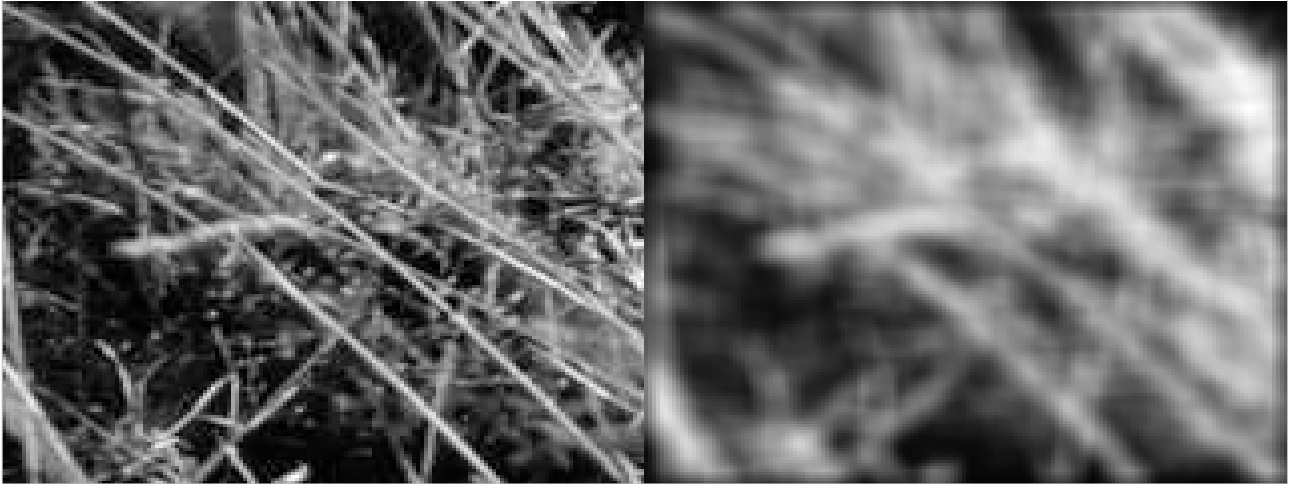
Credit: S. Seitz

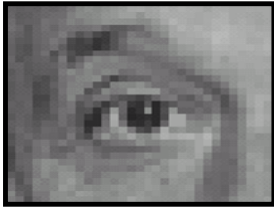
- What is the result of such filtering?
 - Neighborhood average
 - Low pass filter
- Basically sharp transitions are reduced
 - blurring
- **Mean/Box filter smoothing**

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

Il box filter è uno dei più usati (e semplici)





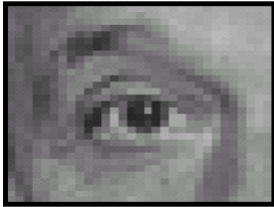
Original

0	0	0
0	1	0
0	0	0

?

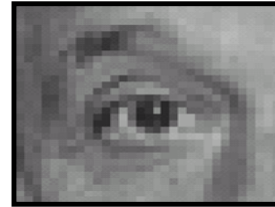
Slide credit: David Lowe (UBC)

È un esempio non molto sensato ma solo per far capire il meccanismo



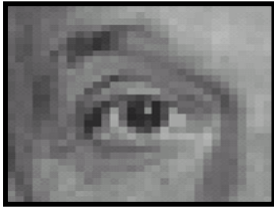
Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Slide credit: David Lowe (UBC)

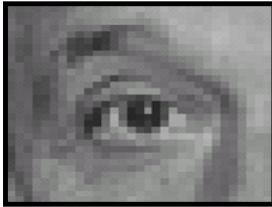


Original

0	0	0
0	0	1
0	0	0

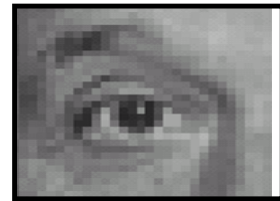
?

Slide credit: David Lowe (UBC)



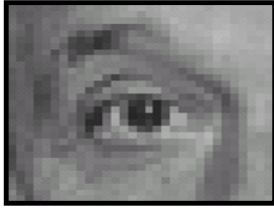
Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

Slide credit: David Lowe (UBC)



Original

0	0	0
0	2	0
0	0	0

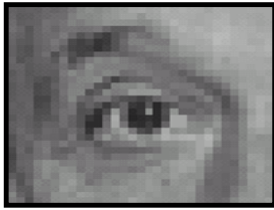
-

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

Slide credit: David Lowe (UBC)



Original

0	0	0
0	2	0
0	0	0

■

$\frac{1}{9}$

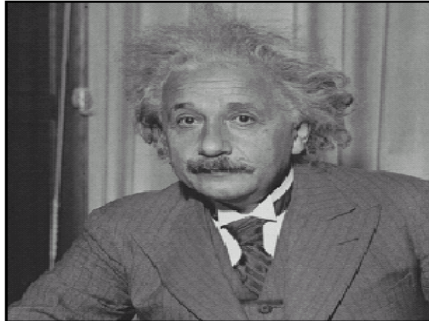
1	1	1
1	1	1
1	1	1



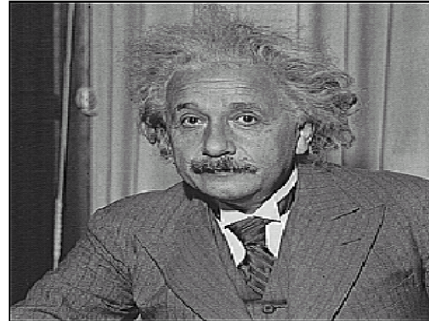
Sharpening filter

- Accentuates differences with local average
- High pass filtering

Slide credit: David Lowe (UBC)

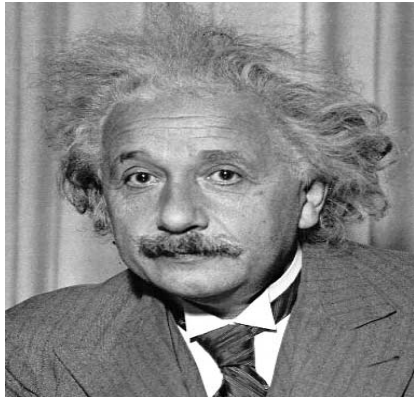


before



after

Slide credit: David Lowe (UBC)



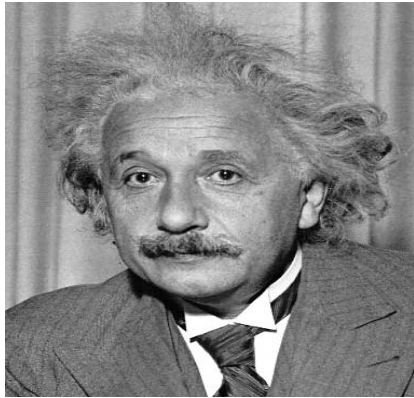
1	0	-1
2	0	-2
1	0	-1

Vertical
Sobel



**Vertical Edge
(absolute value)**

Slide credit: David Lowe (UBC)



1	2	1
0	0	0
-1	-2	-1

Horizontal
Sobel



**Horizontal Edge
(absolute value)**

Slide credit: David Lowe (UBC)

Horizontal Gradient

0	0	0
-1	0	1
0	0	0

or

-1	0	1
----	---	---

Vertical Gradient

0	1	0
0	0	0
0	-1	0

or

-1
0
1

Slide credit: David Lowe (UBC)

Il kernel può contenere valori negativi. E anche valori non interi.

Non è un caso limite si usa spesso.

E quindi l'immagine risultante? Non avevamo detto che usavamo solo valori non negativi?

- We can have negative values
- Generally pixel channels are unsigned
- How to handle them?
 - Clamping: put negative values as 0
 - Abs: consider absolute values only
 - Shift: add a value (may reduce interval)
 - Use them: adapt internal representation to handle them

Se uso dei “pesi” negativi potenzialmente potrei ottenere valori negativi anche nel risultato... Come li gestisco? Innanzitutto non è detto abbia senso doverli adattare. Il risultato può essere a sua volta una immagine ma anche no.

Diciamo che spesso però è comodo sia una immagine soprattutto per visualizzazione (altrimenti non potevo mostrarvi gli “edge” del buon Albert...)

Qui vi si elencano alcune strategie. A parte la quarta opzione (che va letta come “me li tengo”) nelle altre spesso perdo informazione.

- **Some** kernel filters are **separable**
- $n \times n$ kernel in a single convolution
 - $O(M \times M \times n \times n)$
- $1 \times n$ and $n \times 1$ convolutions
 - $O(M \times M \times (n+n))$
- Mean filtering example:

$$\frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * \frac{1}{3} [1 \quad 1 \quad 1] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Le operazioni di convoluzione sono onerose. In quanto devo ripetere per ciascun pixel dell'immagine ($M \times M$) e per ciascuno di questi guardare il vicinato (che dipende dalle dimensioni del kernel $n \times n$)

Molti filtri sono però “separabili”, come nell'esempio in esame invece di applicare un singolo kernel 3×3 ne applico due, uno 1×3 e un secondo 3×1 . Il risultato è lo stesso ma riduco le operazioni da fare di $1/3$ (e sarebbe molto di più per kernel più “grossi”)

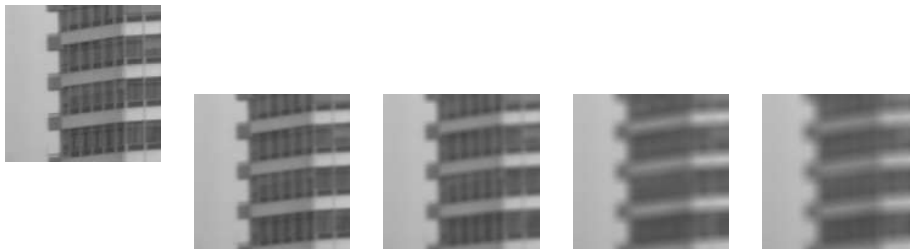
- Image can be affected by noise
 - Sensor
 - Transmission
 - Lossy compressions
 - Filtering can enhance noise (derivative operations)
- Different kind of noises
 - White noise
 - Salt&Pepper
 - ...
- Smoothing is often used to “reduce” it



- Averaging neighborhood can smooth images
- The bigger the kernel size, the augmented the smooth effect
 - Balancing act
 - Examples with 3x3, 5x5, 9x9, and 11x11 kernels

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1



Lo abbiamo già visto, è in pratica una media su una finestra $n \times n$. Aumentando n aumenta anche l'effetto smoothing. Eliminiamo dei dettagli. È un filtro passa basso.

- Pros
 - Easy implementation
- Cons:
 - Weight of each pixel is the same
 - Not good for specific noise effects
 - Introduce high frequency artifacts

Una media è semplice da implementare. Però di fatto non tengo conto della distanza dei pixel di vicinato da quello su cui il kernel è “centrato”. Oltretutto il kernel è tipicamente quadrato. I pixel negli angoli pesano uguale ai pixel sui bordi...



Noisy Image



“Ideal” Filtering

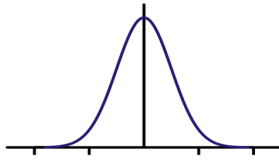


Mean Filtering result 👎

Non risolve tutte le situazioni, ad esempio il rumore “sale e pepe” lo vorrei direttamente eliminare e non tenerne conto ai fini media.

- Mean filtering kernel use the same weight for each pixel
 - But the distance is different
- Gaussian filtering gives more weight to the central pixel and:
 - The farther away the neighbors, the smaller their contribution
 - A gaussian is used

Una media è semplice da implementare. Però di fatto non tengo conto della distanza dei pixel di vicinato da quello su cui il kernel è “centrato”. Oltretutto il kernel è tipicamente quadrato. I pixel negli angoli pesano uguale ai pixel sui bordi...

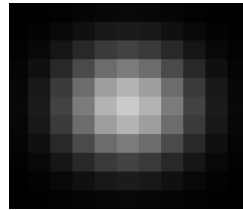


$$g_{\sigma}(u,v)=\frac{1}{2\pi\sigma_2}e^{-\frac{u^2+v^2}{2\sigma^2}}$$



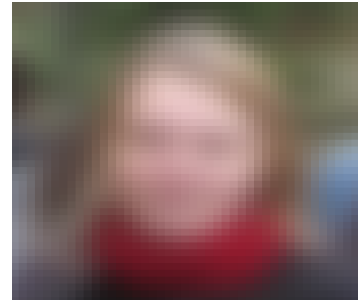
Input image f

*



Filter g

=

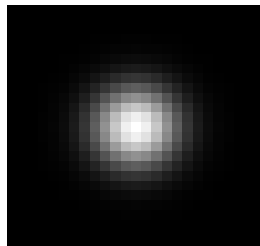
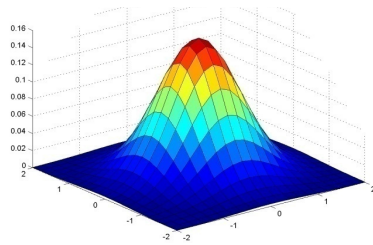


Output image h

U e v sono le “coordinate” dei pesi del kernel con 0,0 nel centro. All’aumentare del loro valore diminuisce $g()$



Spatially-weighted average



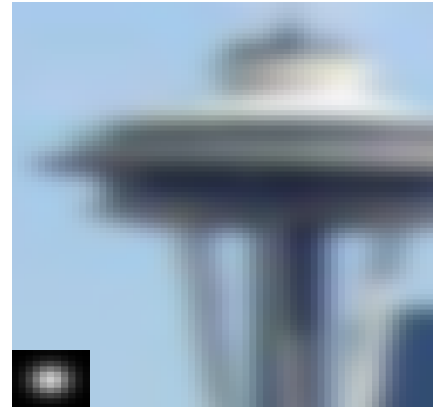
0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$
(normalized values)

$$g_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

Slide credit: Christopher Rasmussen

u e v sono le “coordinate” dei pesi del kernel con 0,0 nel centro. All’aumentare del loro valore diminuisce g()
 Però una gaussiana si estende all’infinito, il kernel no.
 Quindi, oltre ad applicare la formula devo normalizzare.
 In quel kernel 5x5 i pesi non sono quelli direttamente calcolati grazie alla formula ma sono stati “aggiustati” di modo da ottenere 1 come loro somma (come lo sarebbe l’integrale della gaussiana da $-\infty$ a $+\infty$)



Nel caso del mean box si possono notare vignettature verticali e orizzontali mentre nel caso del gaussiano non ci sono

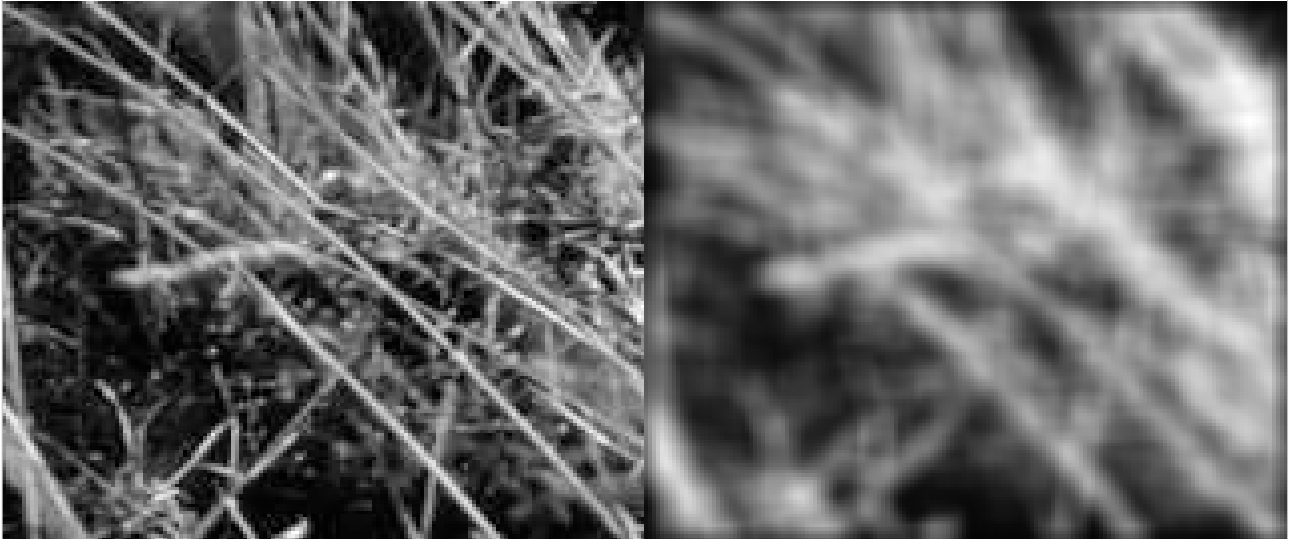
SMOOTHING: Gaussian vs Mean

UNIVERSITÀ
DI PARMA



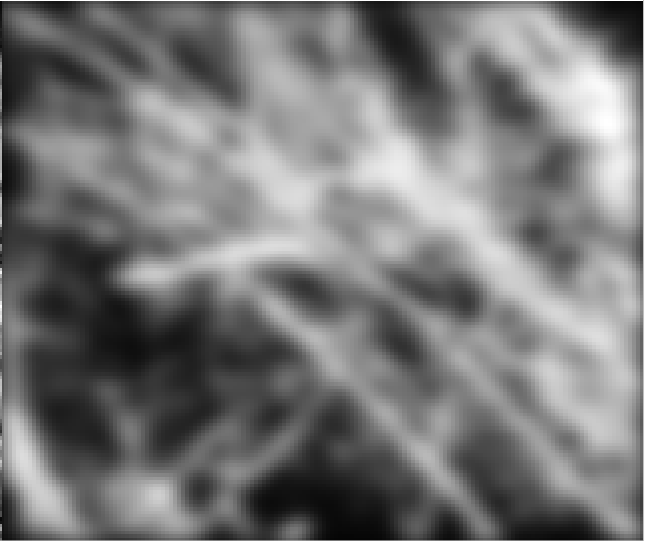
Slide credit: Robert Collins

SMOOTHING: mean filtering artifacts

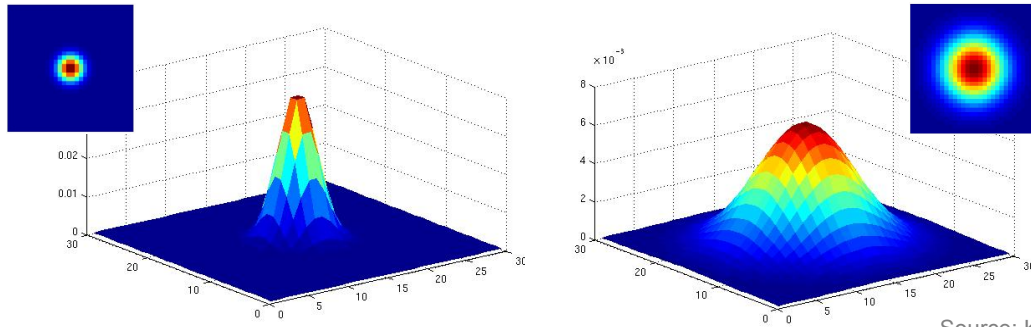


SMOOTHING: Gaussian

UNIVERSITÀ
DI PARMA



What parameters matter here?
Variance of Gaussian: determines
extent of smoothing

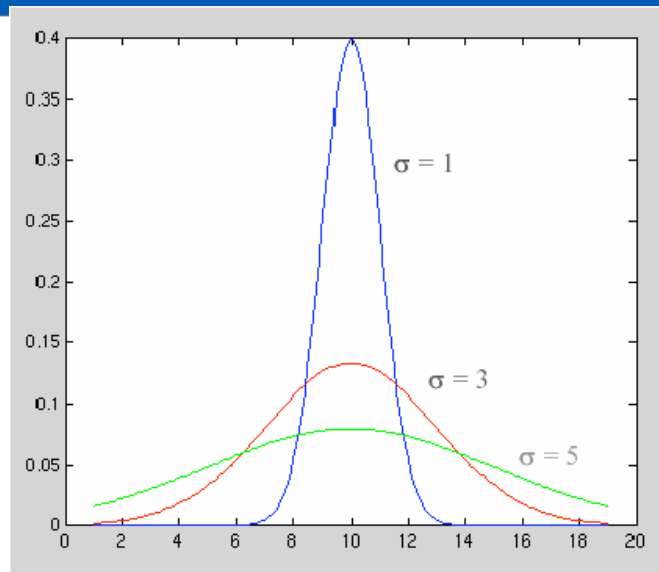


Source: K. Grauman

Il sigma mi determina l'andamento della gaussiana

What parameters matter here?

Variance of Gaussian:
determines extent of
smoothing

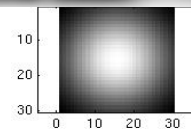
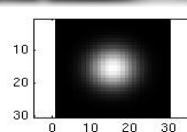
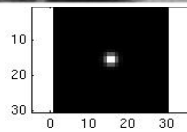


Il sigma mi determina l'andamento della gaussiana.
Aumentarlo significa aumentare il peso dei pixel
“lontani”, diminuirlo significa aumentare l'importanza
del pixel “centrale”

SMOOTHING: Gaussian



Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel, and controls the amount of smoothing.



Source: K. Grauman

Look at edges... Do you see something?

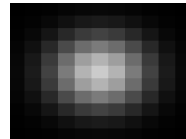


$$\mathcal{G}_\sigma = \mathcal{G}_\sigma^x * \mathcal{G}_\sigma^y$$

$$\mathcal{G}_\sigma^x(x, y) = \frac{1}{Z} e^{-\frac{x^2}{2\sigma^2}}$$

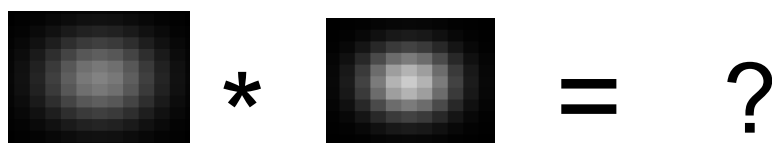
$$\mathcal{G}_\sigma^y(x, y) = \frac{1}{Z} e^{-\frac{y^2}{2\sigma^2}}$$

Gaussian filter is **separable**: compute in **horizontal** direction, followed by the **vertical** direction.



Faster!

Anche il filtro gaussiano è separabile



$$f * \mathcal{G}_\sigma * \mathcal{G}_{\sigma'} = f * \mathcal{G}_{\sigma''}$$

$$\sigma'' = \sqrt{\sigma^2 + \sigma'^2}$$

Cascade Gaussians: we can use smaller kernels

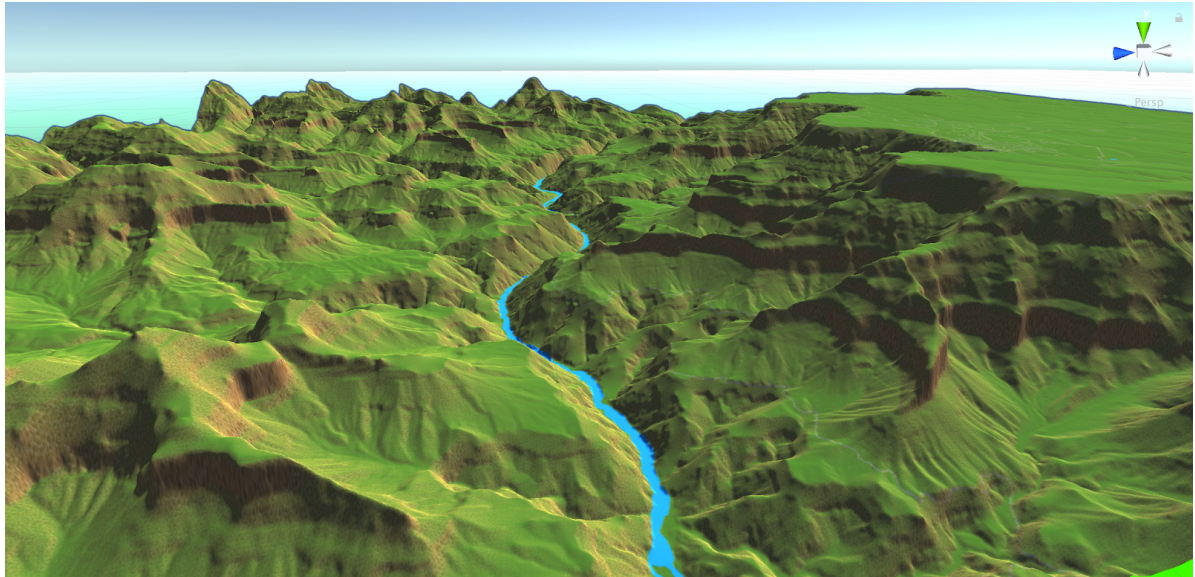
Con sigma alti avrebbe anche senso usare kernel più grandi (altrimenti si cade nel letterbox). Però è costoso computazionalmente.

Vediamo cosa succede se combino due convoluzioni consecutive con due sigma diversi (ma ovviamente la formula è la stessa con sigma uguali)

Questo mi permette di fare convoluzioni con kernel più piccoli e ottenere lo stesso risultato di un kernel più grande (al netto di qualche approssimazione dovuta alle dimensioni finite dei kernel)

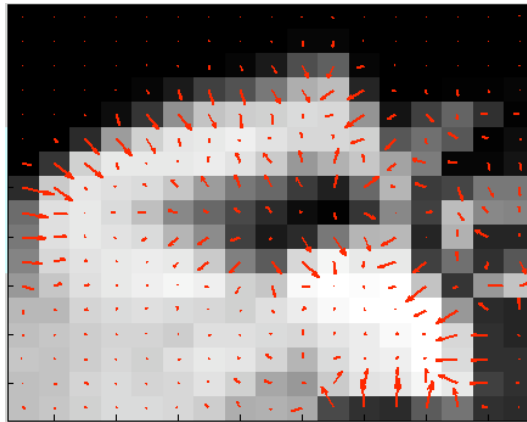


- Uniform areas → low frequency
- Edges or sharp transitions → high frequency
- Sharpening is obtained enhancing higher frequencies
 - It can be seen as opposite wrt smoothing



- We can see images as 2D “surfaces” where pixel “intensity” encodes elevation
- Therefore we can use concepts like:
 - Uphill/Downhill
 - Peaks/Valleys
 - Discontinuities/Steepness
- Do you remember we can see images as functions?
 - $I(x,y)$
- For each (x,y) what function can say us something about local intensity variations?
 - Gradient

$$\nabla I(x,y) = \begin{bmatrix} \frac{\partial I(x,y)}{\partial x} \\ \frac{\partial I(x,y)}{\partial y} \end{bmatrix}$$



Credits: Robert Collins

- For each pixel we have a vector
 - Magnitude
 - Direction
- Images are discrete functions
 - How we can compute numerical derivatives?

$$\frac{\partial I(x, y)}{\partial x} = \lim_{h \rightarrow 0} \frac{I(x+h, y) - I(x, y)}{h}$$

Forward difference is usually used but centered difference is more accurate when it comes to discrete case



- Taylor Series Expansion can be used
- Precision of the following formula is $O(h^2)$

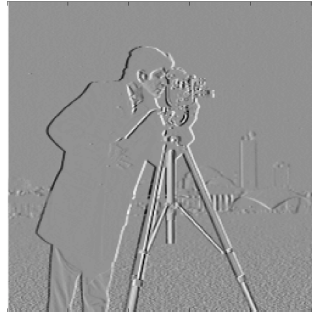
$$\frac{\partial I(x, y)}{\partial x} \approx \frac{I(x+h, y) - I(x-h, y)}{2h} \quad \frac{\partial I(x, y)}{\partial y} \approx \frac{I(x, y+h) - I(x, y-h)}{2h}$$



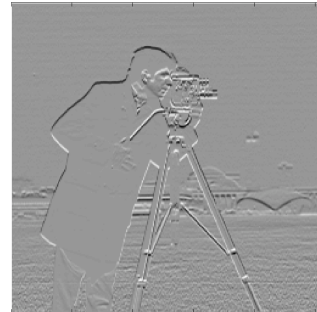
$$I(x, y)$$



$$I_x = \frac{I(x+1, y) - I(x-1, y)}{2}$$



$$I_y = \frac{I(x, y+1) - I(x, y-1)}{2}$$



Credits: Robert Collins



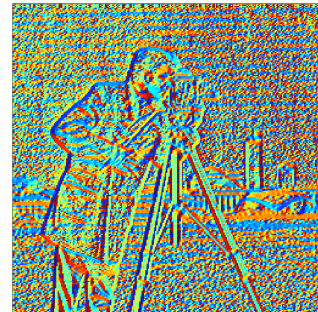
$$I(x, y)$$



$$\sqrt{I_x^2 + I_y^2}$$



$$\text{atan2}(I_y, I_x)$$



Credits: Robert Collins

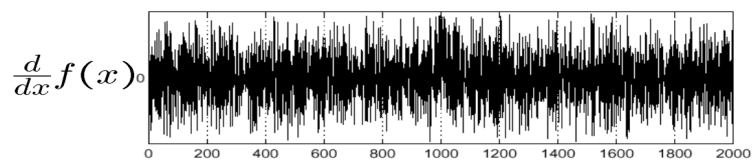
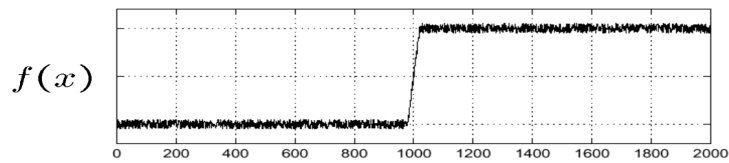
- I_x and I_y can be easily computed using simple operators

$$I_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} * I \quad I_x = \begin{bmatrix} +1 & 0 & -1 \end{bmatrix} * I$$

- Problem: noise



Consider a single image row



Finding the “edge” is not so easy...

Source: A. Bobick

- Usually we want to find *strong* gradient variations (edges)
- How to remove noise? → smoothing

$$I_y = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} * ([1 \ 1 \ 1] * I) \quad I_x = [+1 \ 0 \ -1] * \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * I$$

- Usually we want to find *strong* gradient variations (edges)
- How to remove noise? → smoothing

$$I_y = \begin{bmatrix} +1 & 0 & -1 \\ +1 & 0 & -1 \\ +1 & 0 & -1 \end{bmatrix} * I \quad I_x = \begin{bmatrix} +1 & +1 & +1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} * I$$

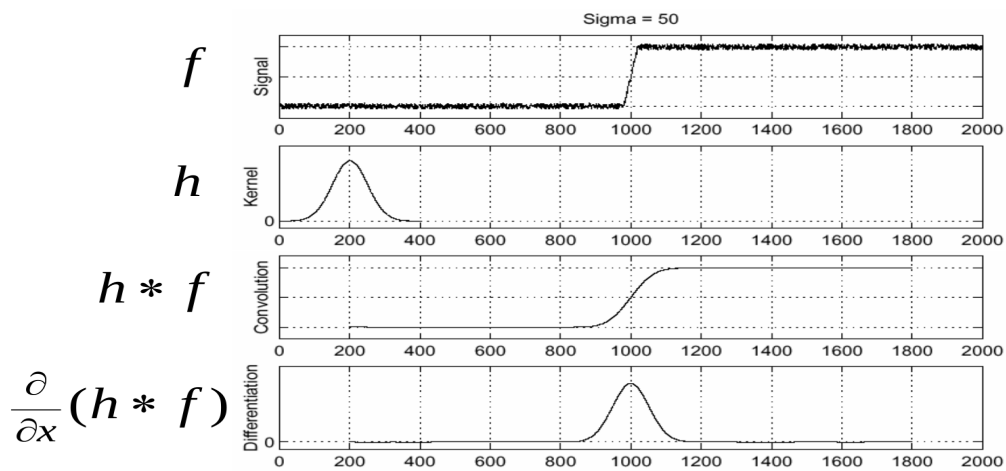
- Prewitt

- As seen for smoothing for prewitt operators we have the same weight
- **Sobel** operators slightly improves this

$$I_y = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * I \quad I_x = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * I$$

- For smoothing we also saw a gaussian approach

Gradient filtering



Where is the maximum gradient?

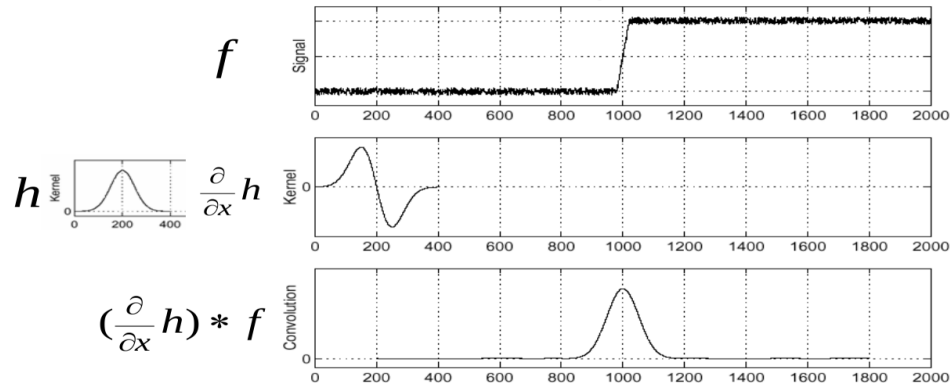
Source: A. Bobick



- Introducing Prewitt/Sobel we distributed the convolution
 - Namely we applied the derivative operator to smoothing one
- Derivative theorem of convolution

$$\frac{\partial}{\partial x}(f * g) = \left(\frac{\partial}{\partial x} f \right) * g = f * \left(\frac{\partial}{\partial x} g \right)$$

- This saves us one operation: $\frac{\partial}{\partial x}(h * f) = (\frac{\partial}{\partial x}h) * f$



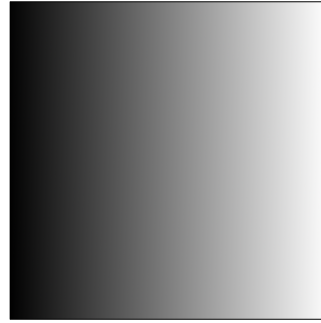
And if we want to find local maxima?

Source: A. Bobick



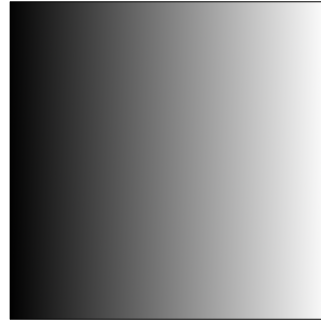
- 1st derivative filters
- Where is the edge?
 - Magnitude peaks

$$\sqrt{\frac{\partial}{\partial x} I(x, y)}$$



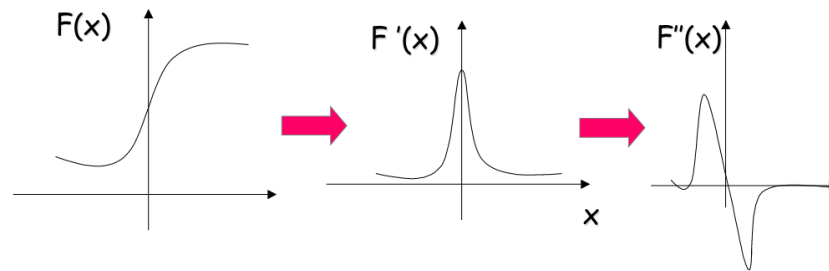
- 1st derivative filters
- Where is the edge?
 - Magnitude peaks

$$\sqrt{I_x^2 + I_y^2} > Th$$





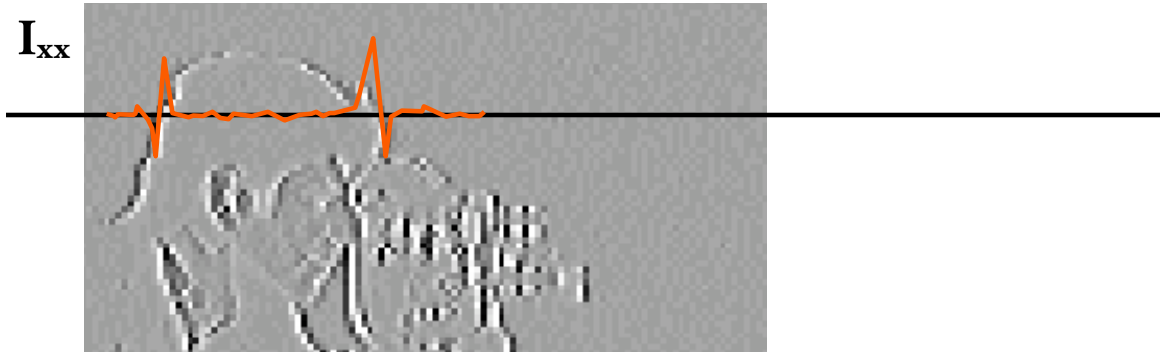
- Using 1st derivative filters maxima detect edges
 - Not simple to exactly find them
- What about 2nd derivative filters?



Source: R. Collins



- Pro: easy localization of edges
- Cons: 2nd derivative is 0 also when 1st one is 0



Source: R. Collins

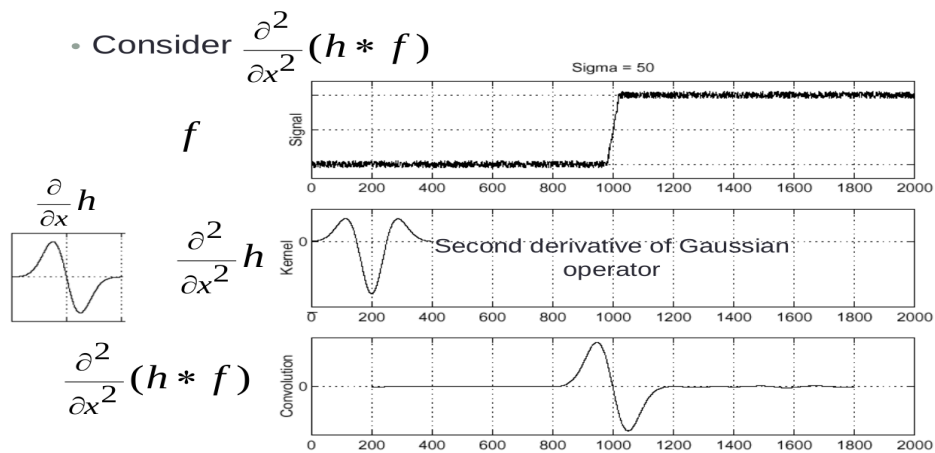
Soluzione: considero gli zeri solo dove la derivata prima ha sufficiente magnitude

- Laplacian operator ∇^2 can be used to combine I_{xx} and I_{yy}

$$\nabla^2 I(x, y) = I_{xx}(x, y) + I_{yy}(x, y) \approx \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} * I$$

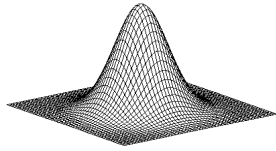


Source: R. Collins



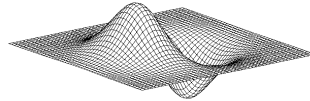
The maxima is where we have a zero crossing

Source: A. Bobick



Gaussian

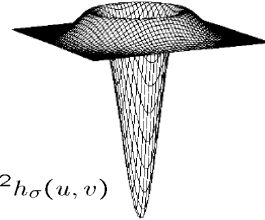
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

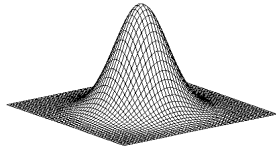
Laplacian of Gaussian
or LoG filter



$$\nabla^2 h_{\sigma}(u, v)$$

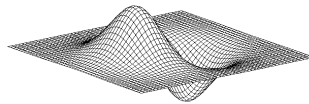
∇^2 is the Laplacian operator (sum of 2nd derivatives):

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$



Gaussian

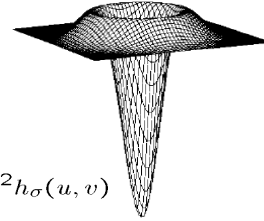
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

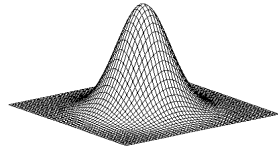
$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian
or LoG filter



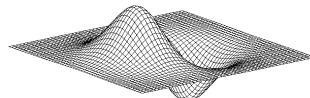
$$\nabla^2 h_{\sigma}(u, v)$$

$$\begin{aligned} &\text{Laplacian(Gaussian * f)} \\ &= \\ &\text{LoG * f} \end{aligned}$$



Gaussian

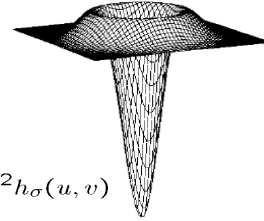
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian
or LoG filter



$$\nabla^2 h_{\sigma}(u, v)$$

$$\nabla^2 G(x, y; \sigma) = \left(\frac{x^2 + y^2}{\sigma^4} - \frac{2}{\sigma^2} \right) G(x, y; \sigma)$$

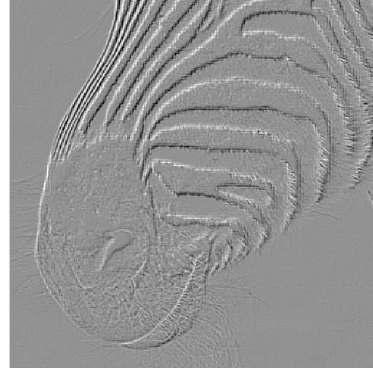
Often approximated
by

0	1	0
1	-4	1
0	1	0

C'è anche la versione con tutti 1 e -8 oppure la negativa



- Band pass filter
 - Suppress low frequencies (2^{nd} derivative) but also high ones (gaussian)



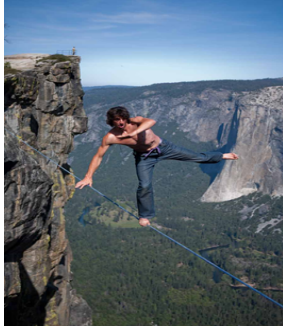
Source: R. Collins

- Used as “edge” detector, extract “raw” zero crossings:
 - Different sigmas (2 & 4)
 - Like a “equal elevation” on a map

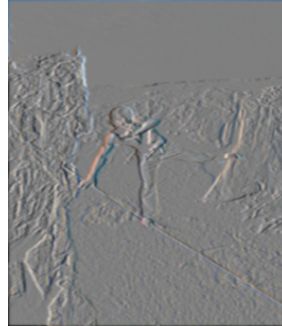


Source: R. Collins

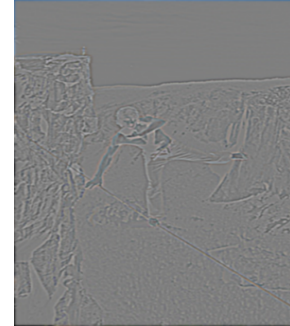
Ricavata da lecture11 di Collins
Analizza altri usi come l'individuazione di blob e
tecniche di compressione lossy in cui un LoG viene
applicato ad una piramide e guadagno rispetto a
comprimere l'immagine intera



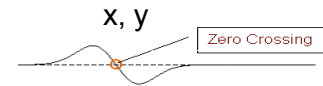
Original



Derivata prima x



Derivata seconda

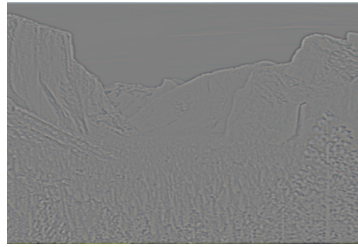


Filtro di sharpening

$$\textit{Sharpen}(x, y) = f(x, y) - \alpha(f * \nabla^2 \mathcal{G}_\sigma(x, y))$$



Originale

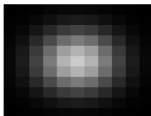
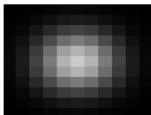


Derivata seconda



Sharpened

$$f * g * g' = f * h \text{ for some } h$$

<table border="1"> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>-1</td><td>0</td><td>1</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr> </table>	0	0	0	0	0	0	0	0	0	0	0	-1	0	1	0	0	0	0	0	0	0	0	0	0	0	*		=	
0	0	0	0	0																									
0	0	0	0	0																									
0	-1	0	1	0																									
0	0	0	0	0																									
0	0	0	0	0																									
<table border="1"> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>-1</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>-1</td><td>4</td><td>-1</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>-1</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr> </table>	0	0	0	0	0	0	0	-1	0	0	0	-1	4	-1	0	0	0	-1	0	0	0	0	0	0	0	*		=	
0	0	0	0	0																									
0	0	-1	0	0																									
0	-1	4	-1	0																									
0	0	-1	0	0																									
0	0	0	0	0																									

It's also true:

$$f * (g * h) = (f * g) * h$$

$$f * g = g * f$$

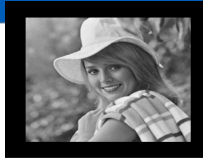


- We now should have understood basic convolution principles
- We can add further complexity
 - Border issues
 - Padding
 - Stride



- How we deal with border/corner pixels in a convolution?
 - Actually the same issue for other operations that involve a mask/kernel
- Basically two approaches
 - Result of the processing is smaller than original image
 - Same size with border/corner pixels with *unknown* value

Borders issue

[illegible][illegible]

0	10	20	30	30	30	20	10
0	20	40	60	60	60	40	20
0	30	60	90	90	90	60	30
0	30	50	80	80	90	60	30
0	30	50	80	80	90	60	30
0	20	30	50	50	60	40	20
10	20	30	30	30	30	20	10
10	10	10	0	0	0	0	0

[illegible]



- **Padding** is a technique to “enlarge” images adding a given amount of pixels to each image border
- The most common approach is the **zero padding**
 - Added pixels are set to 0



- Padding is often use before a convolution to obtain same size result
- The “extra” pixels can be set using different policies
 - Again: zero padding is the most used



black



fixed



periodic



reflected

- For a 4x4 image

20	128	60	100
50	66	72	20
73	98	65	89
20	170	190	200

- Zero padding 1 → output size is **6x6**

0	0	0	0	0	0
0	20	128	60	100	0
0	50	66	72	20	0
0	73	98	65	89	0
0	20	170	190	200	0
0	0	0	0	0	0

- When we use a 3x3 kernel we obtain a 4x4 resulting image

- Zero padding 2 → output size is **8x8**

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	20	128	60	100	0	0
0	0	50	66	72	20	0	0
0	0	73	98	65	89	0	0
0	0	20	170	190	200	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

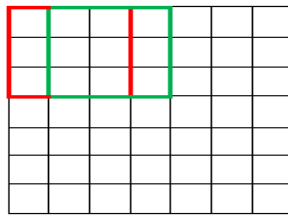


- The stride is a parameter used to set kernel movements on the image
- Since now we assumed that kernel is horizontally and vertically moved on each row/column $\rightarrow$ stride is 1
- We can move a kernel skipping some pixel $\rightarrow$ stride > 1
- A stride > 1 leads to subsampling images!
- Moreover we can have different strides for rows/columns

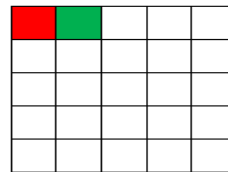


- Stride = 1 → “normal convolution”. Kernel is moved on every pixel (vertically & horizontally)
- I.e for 7x7 image, a 3x3 kernel and a stride value as 1 we will obtain a **5x5 image**

7 x 7 Input Volume



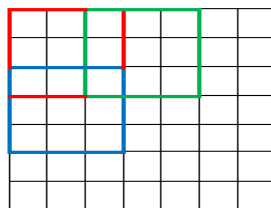
5 x 5 Output Volume





- Stride = 2 $\rightarrow$ kernel is moved skipping 1 column for each horizontal move and 1 row for each vertical move
- I.e for 7x7 image, a 3x3 kernel and a stride value as 1 we will obtain a **3x3 image**

7 x 7 Input Volume



3 x 3 Output Volume



- Given a image featuring n columns
- Using a kernel with f columns, with padding p and stride s , the number of columns of the output image is:

$$n_{new} = \lfloor \frac{n + 2 * p - f}{s} + 1 \rfloor$$

- The result must be the largest integer value equal or smaller than n_{new}
- The same formula can be used for computing the new number of rows



- Since now we illustrate the so-called **linear filters**
 - The result is a linear sum of neighborhood values
 - Linear filters can easily be combined:
 - The result of the processing of the sum of 2 signals can be obtained as the sum of the processing of the 2 signals

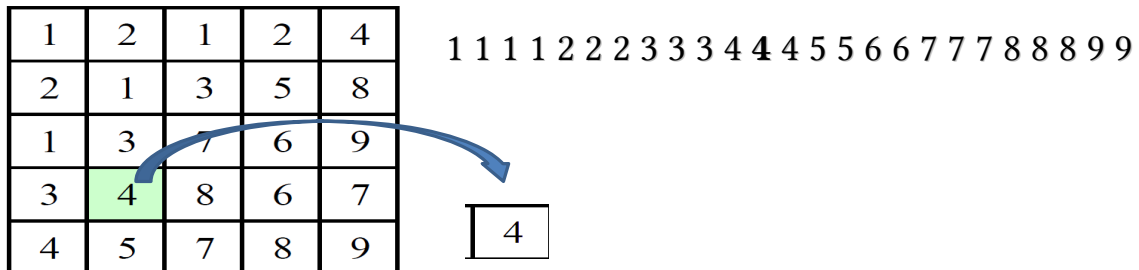
$$S(\alpha * f[x, y] + \beta * g[x, y]) = \alpha * S(f[x, y]) + \beta * S(g[x, y]) \text{ for all } \alpha, \beta \in \mathbb{R}$$

- Linear filters can be easily combined and can also be analyzed in the frequency domain (Fourier transform) but they are not always the most efficient solution



- Median pooling
- Max/Min pooling
- *Average pooling*
- Bilateral filter

- For each neighborhood we select the median value



- Is a smoothing operation
- Can cope with salt & pepper noise
- Sorting needed → computationally expensive

Source: Szeliski



Shot noise



5x5 average



5x5 median

Source: Collins

- Select the largest/smallest value in the neighborhood

20	128	60	100
50	66	72	20
73	98	65	89
20	170	190	200

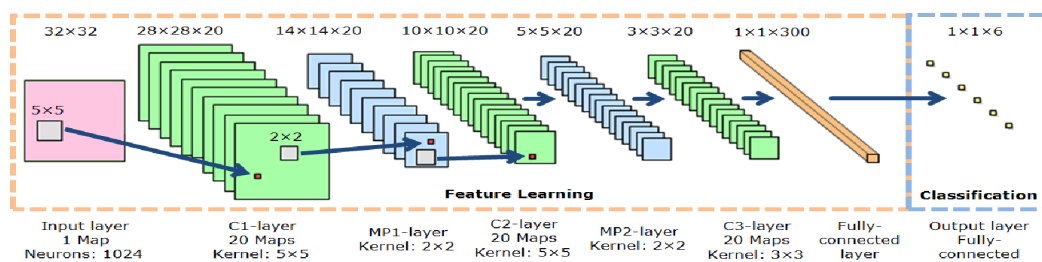
128	128	100
98	98	89
170	190	200

- Can be used to selectively remove salt & pepper noise



- We already saw this guy!
- Box filter (linear)

- Convolutioni con stride e max pooling sono gli elementi base di una **Convolutional Neural Network (CNN)**:

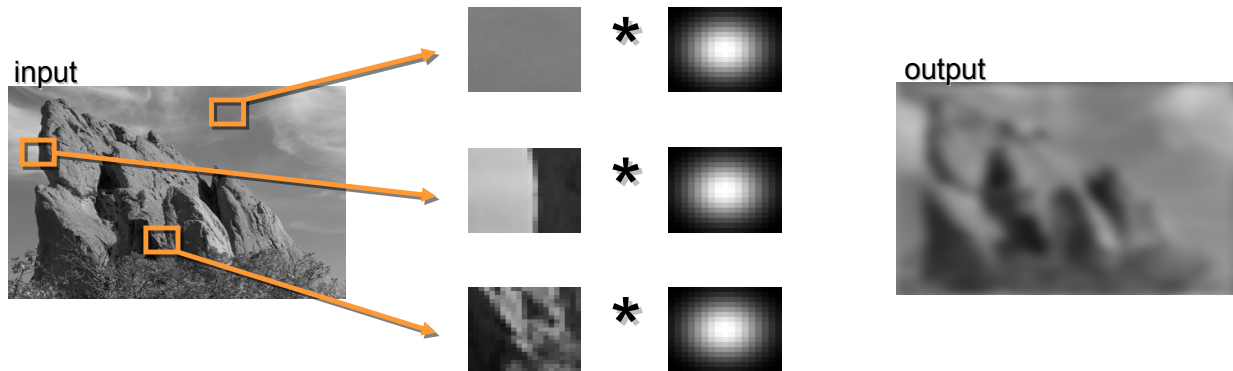


La differenza è che le convoluzioni non sono staticamente decise dal programmatore ma in pratica è l'addestramento che le stabilisce ovvero dinamicamente



- Smoothing vs Edges
 - Smoothing is mainly used for noise reduction
 - Anyway also edges can be affected...
- Gaussian filters do not depend on image content
 - Strong gradient variations are treated like uniform areas

Bilateral filter



Slides courtesy of Sylvian Paris

- Bilateral filter exploits a specific weight for adapting a gaussian kernel to image content
- $w(i,j,k,l) \rightarrow$ weighting coefficient

$$g(i,j) = \frac{\sum_{k,l} f(i+k, j+l) w(i,j,k,l)}{\sum_{k,l} w(i,j,k,l)}$$

Source: Szeliski

G immagine risultante

F immagine sorgente

K and l are absolute coordinates nel kernel

Normalmente il kernel NON dipende da i,j

- The weighting coefficient can be obtained as the composition of 2 kernels:
 - Gaussian filtering → **Domain** Kernel
 - Independent from image content
 - Weight correction → **Range** Kernel
 - Dependent on (local) image content
- Bilateral filtering is space variant since kernel depends on pixel value
 - Non linear filtering

- The domain kernel is a Gaussian Kernel

$$d(i, j, k, l) = e^{-\left[\frac{(i-k)^2 + (j-l)^2}{2\sigma_d^2} \right]}$$

- $d(i, j, k, l)$ does not depend on image content

Source: Szeliski

- The range kernel is data dependent

$$r(i, j, k, l) = e^{-\left[\frac{\|f(i, j) - f(i+k, j+l)\|^2}{2\sigma_r^2} \right]}$$

- $r(i, j, k, l)$ uses the **vector distance** between center pixel and neighboring one!

Source: Szeliski

E elevato alla 0 è 1

- The overall kernel is

$$w(i, j, k, l) = e^{-\frac{(i-k)^2 + (j-l)^2}{2\sigma_d^2} - \frac{\|f(i, j) - f(i+k, j+l)\|^2}{2\sigma_r^2}}$$

- $d(i, j, k, l)$ uses the **vector distance** between center pixel and neighboring one!

Source: Szeliski

Bilateral filter

0.1	0.2	0.0	200	213	222
0.0	0.2	0.3	201	223	247
0.2	0.7	0.1	210	201	233
0.4	0.3	0.5	220	216	235
0.3	0.4	0.3	225	250	221
0.6	0.5	0.6	215	243	200

range kernel with $\sigma=5$

0.999	0.999	0.0
0.992	1.0	0.0
0.999	0.996	0.0

domain kernel $\sigma=2$

0.77	0.88	0.77
0.88	1.0	0.88
0.77	0.88	0.77

w

0.76	0.87	0.0
0.872	1.0	0.0
0.76	0.87	0.0

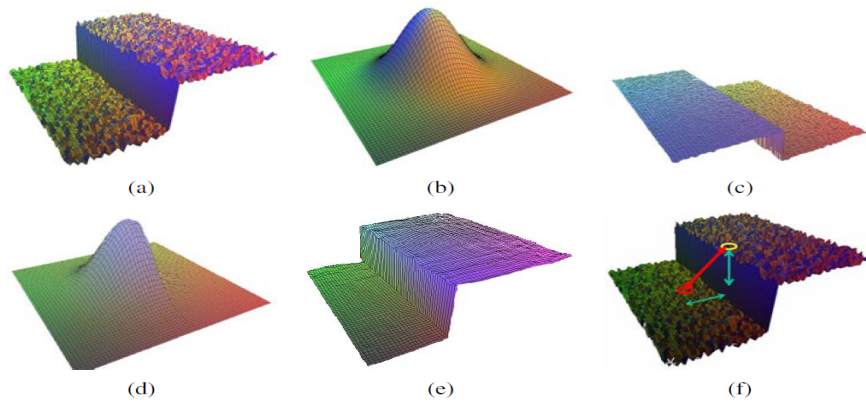
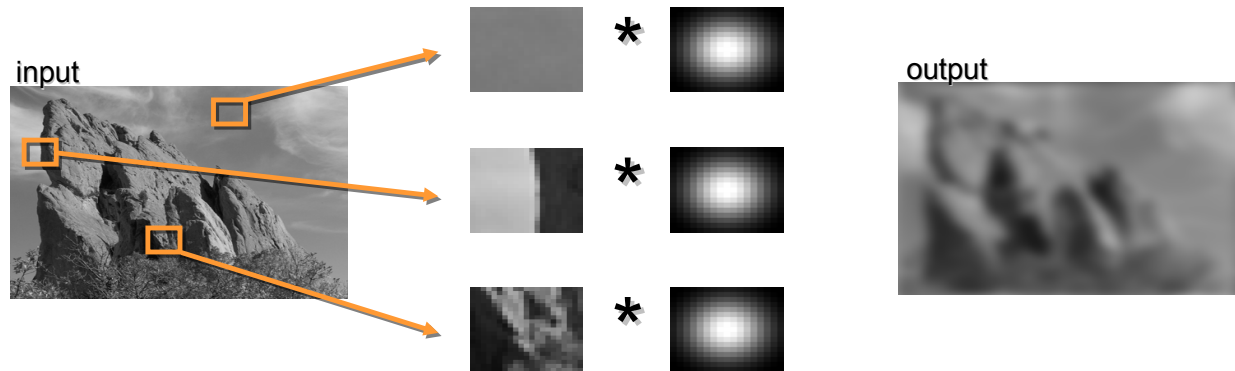


Figure 3.20 Bilateral filtering (Durand and Dorsey 2002) © 2002 ACM: (a) noisy step edge input; (b) domain filter (Gaussian); (c) range filter (similarity to center pixel value); (d) bilateral filter; (e) filtered step edge output; (f) 3D distance between pixels.

Source: Szeliski

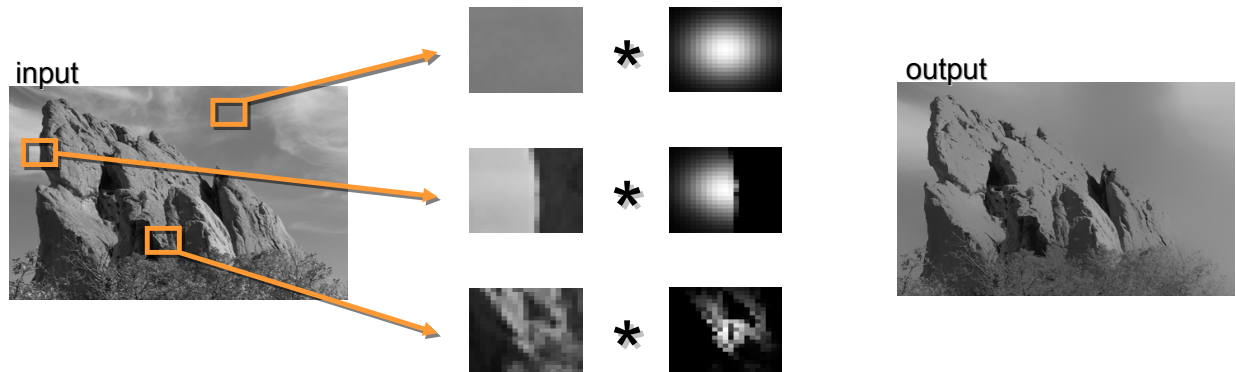
Bilateral filter



Slides courtesy of Sylvian Paris



Maintains edges when blurring!



The kernel shape depends on the image content.

Slides courtesy of Sylvian Paris

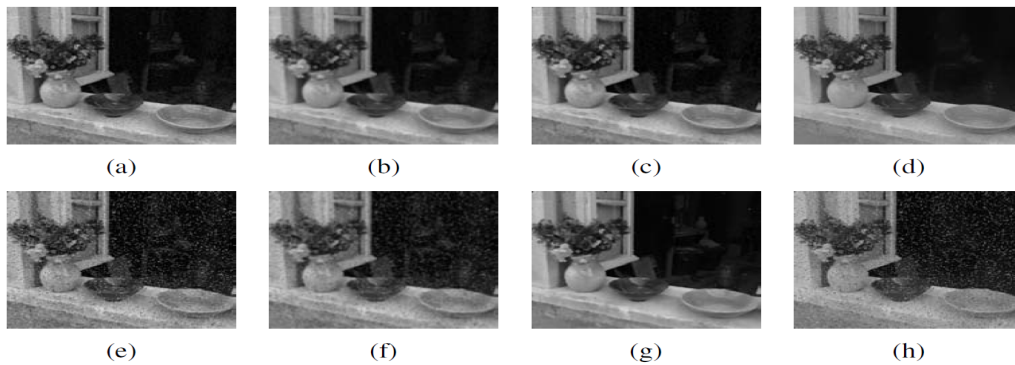


Figure 3.18 Median and bilateral filtering: (a) original image with Gaussian noise; (b) Gaussian filtered; (c) median filtered; (d) bilaterally filtered; (e) original image with shot noise; (f) Gaussian filtered; (g) median filtered; (h) bilaterally filtered. Note that the bilateral filter fails to remove the shot noise because the noisy pixels are too different from their neighbors.

Szeliski

- Often convolutions imply to repeatedly compute the sum of a region.

$$S(i, j) = \sum_{k=0}^i \sum_{l=0}^j F(k, l)$$

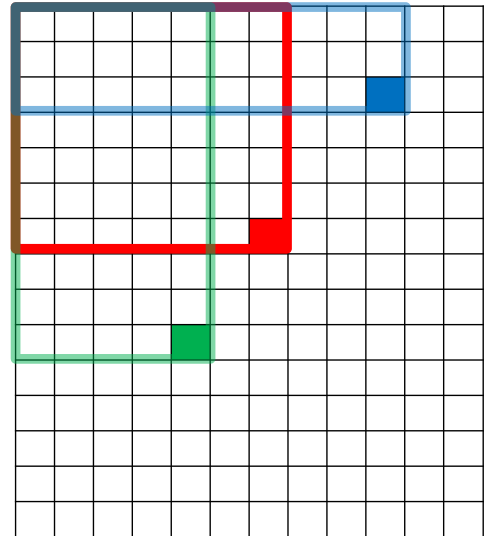
- This is an expensive operation $\rightarrow O(n^2)$
- The **integral image** can lead to $O(n)$
- Also known as **summed area table**

243	239	240	225	206	185	188	218	211	206	216	225
242	239	218	110	67	31	34	152	213	206	208	221
243	242	123	58	94	82	132	77	108	208	208	215
235	217	115	212	243	236	247	139	91	209	208	211
233	208	131	222	219	226	196	114	74	208	213	214
232	217	131	116	77	150	69	56	52	201	228	223
232	232	182	186	184	179	159	123	93	232	235	235
232	236	201	154	216	133	129	81	175	252	241	240
235	238	230	128	172	138	65	63	234	249	241	245
237	236	247	143	59	78	10	94	255	248	247	251
234	237	245	193	55	33	115	144	213	255	253	251
248	245	161	128	149	109	138	65	47	156	239	255
190	107	39	102	94	73	114	58	17	7	51	137
23	32	33	148	168	203	179	43	27	17	12	8
17	26	12	160	255	255	109	22	26	19	35	24

Fare esempio media box/filter



- Each pixel (x,y) contains the sum of other pixels contained in the rectangle $(0,0)-(x,y)$



4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4				

Calcolo semplice: nell'immagine integrale sommo
cella sopra + cella a sinistra - cella in alto a sinistra +
cella in immagine originale

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7			

Calcolo semplice: nell'immagine integrale sommo
cella sopra + cella a sinistra - cella in alto a sinistra +
cella in immagine originale

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14		

Calcolo semplice: nell'immagine integrale sommo
cella sopra + cella a sinistra - cella in alto a sinistra +
cella in immagine originale

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	

Calcolo semplice: nell'immagine integrale sommo
cella sopra + cella a sinistra - cella in alto a sinistra +
cella in immagine originale



4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6				

Calcolo semplice: nell'immagine integrale sommo
cella sopra + cella a sinistra - cella in alto a sinistra +
cella in immagine originale

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14			

Calcolo semplice: nell'immagine integrale sommo
cella sopra + cella a sinistra - cella in alto a sinistra +
cella in immagine originale



- A fast way to compute the summed area table is:
 - Initialize integral image I using same values of original image
 - Start from (0,0) and move row by row
 - Update each “cell” as
 - $I(x, y) = I(x, y) + I(x-1, y) + I(x, y-1) - I(x-1, y-1)$

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

Calcolo semplice: nell'immagine integrale sommo
cella sopra + cella a sinistra - cella in alto a sinistra +
cella in immagine originale

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

Calcolo semplice: nell'immagine integrale sommo
cella sopra + cella a sinistra - cella in alto a sinistra +
cella in immagine originale



4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1



4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	50
11	25	43	68	74
12	27	49	77	85
12	29	54	86	95

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	50
11	25	43	68	74
12	27	49	77	85
12	29	54	86	95

Let's try to compute the sum of that area $7+9+1+4+7+2+4+3+2 = 39$

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85 _A
12	29	54	86	95

Consider the same area on the Integral Image and cells A, B, C, D

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85
12	29	54	86	95 _A

The result is $A+B-C-D \rightarrow 85 + 7 - 26 - 27 = 39$

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85 _A
12	29	54	86	95

The result is $A+B-C-D \rightarrow 85 + 7 - 26 - 27 = 39$

- Examples of usage

- Filter box: given a $n \times n$ kernel on a $M \times N$ image

- No integral image, separable kernels $\rightarrow O(M \times N \times 2n)$
 - Computing integral image $\rightarrow O(M \times N \times 2)$

- Haar like features

- Binary patterns used for face detection

